Q7 - FOOD INVENTORY

Augmented Red-Black Tree - Complete Visual Walkthrough

Lecture 5 / Homework Q7. A step-by-step guide with pseudocode, tree figures, calculations and worst-case running times.

1. Data structure

Use an augmented Red-Black Tree (RB tree), with one node per reported date. The key of every node is its date d.

```
FIELD                PURPOSE
date                 The date used as the search key
produced             Food produced on this date
eaten                Food eaten on this date
tossed               Food tossed on this date
subtreeProduced      Total produced in the entire subtree
subtreeEaten         Total eaten in the entire subtree
subtreeTossed        Total tossed in the entire subtree
subtreeCount         Number of reported dates in the subtree
left, right, p, color Normal RB-tree fields
```

For T.nil, all sums and counts are zero. The RB tree height is O(log n). Subtree totals make range averages efficient without visiting each reported date.

2. Example used throughout

```
Date     Produced      Eaten      Tossed
12823       50           30          10
12825       40           35           5
12826       50           30          10
```

Initially, only 12823 and 12825 are recorded. We first insert 12826 with produced=50, then add eaten=30 and tossed=10 to that same date.

3. Helper function: UPDATE(z)

```
UPDATE(z):
    if z == T.nil:
        return

    z.subtreeCount = 1
        + z.left.subtreeCount
        + z.right.subtreeCount

    z.subtreeProduced = z.produced
        + z.left.subtreeProduced
        + z.right.subtreeProduced

    z.subtreeEaten = z.eaten
        + z.left.subtreeEaten
        + z.right.subtreeEaten

    z.subtreeTossed = z.tossed
        + z.left.subtreeTossed
        + z.right.subtreeTossed
```

UPDATE(z) recomputes information for one node using its own value plus previously stored values in its two children. For example, subtreeProduced = own produced + left subtree produced + right subtree produced.

Time: O(1). UPDATE does a fixed number of additions and does NOT traverse its children.

4. UPDATE_FOOD_INVENTORY(T, x, q, d)

This is the full pseudocode. Reports for an existing date and type are added to the previous amount.

```
UPDATE_FOOD_INVENTORY(T, x, q, d):
    z = RB-SEARCH(T, d)

    if z != T.nil:
        if x == PRODUCED:
            z.produced = z.produced + q
        else if x == EATEN:
            z.eaten = z.eaten + q
        else:
            z.tossed = z.tossed + q

        v = z
        while v != T.nil:
            UPDATE(v)
            v = v.p
        return

    z = NEW-NODE()
    z.date = d
    z.produced = 0
    z.eaten = 0
    z.tossed = 0

    if x == PRODUCED:
        z.produced = q
    else if x == EATEN:
        z.eaten = q
    else:
        z.tossed = q

    z.left = T.nil
    z.right = T.nil
    z.color = RED

    z.subtreeCount = 1
    z.subtreeProduced = z.produced
    z.subtreeEaten = z.eaten
    z.subtreeTossed = z.tossed

    RB-BST-INSERT(T, z)

    v = z.p
    while v != T.nil:
        UPDATE(v)
        v = v.p

    RB-INSERT-FIXUP(T, z)
```

During RB-INSERT-FIXUP, when a rotation happens, call UPDATE on the two nodes whose subtrees

changed: first the node that moves down, then the node that moves up.

Step 1 - Tree before adding date 12826

```
                12823 (BLACK)
                       \
                     12825 (RED)

Node        Psum     Esum     Tsum     Count
12823         90       65       15         2
12825         40       35        5         1
```

Why is 12823.subtreeProduced = 90? Its subtree contains both 12823 and 12825, so 50 + 40 = 90.

Step 2 - Search for date 12826

```
UPDATE_FOOD_INVENTORY(T, PRODUCED, 50, 12826)

z = RB-SEARCH(T, 12826)
```

```
               12823 (BLACK)
                       \
                     12825 (RED)
                            \
                            T.nil
```

The date does not exist, so z = T.nil. We skip the existing-date branch and make a new node.

Step 3 - Create and initialize the node

```
z = NEW-NODE()
z.date = 12826
z.produced = 0
z.eaten = 0
z.tossed = 0

if x == PRODUCED:
    z.produced = 50
```

At first all three amounts are zero. Because x=PRODUCED and q=50, we change produced to 50. The new node is NOT yet inside the tree.

Step 4 - Initialize children, color and subtree values

```
z.left = T.nil
z.right = T.nil
z.color = RED
z.subtreeCount = 1
z.subtreeProduced = z.produced
z.subtreeEaten = z.eaten
z.subtreeTossed = z.tossed
```

```
              +-------------------------+
              |  12826 (RED)            |
              |  own P=50 E=0 T=0      |
              |  subtree P=50 E=0 T=0  |
              |  subtreeCount = 1       |
              +-------------------------+
                   /           \
                 T.nil        T.nil
```

Its subtree contains exactly itself, hence subtreeCount = 1 and each subtree sum equals its own daily value.

Step 5 - BST insert

```
RB-BST-INSERT(T, z)

                12823 (BLACK)
                       \
                     12825 (RED)
                             \
                           12826 (RED) <- new node
```

12826 is greater than both existing keys, so it becomes the right child of 12825. Ancestor totals are now stale until we run UPDATE.

Step 6 - Recompute node 12825

```
v = z.p                       // v = 12825
while v != T.nil:
    UPDATE(v)
    v = v.p

UPDATE(12825):
  subtreeProduced = 40 + 50 = 90
  subtreeEaten    = 35 +  0 = 35
  subtreeTossed   =  5 +  0 =  5
  subtreeCount    =  1 +  1 =  2

                12823 (BLACK)
                       \
                  [12825] (RED) Psum=90, Count=2
                             \
                           12826 (RED)
```

We update 12825 first because it is the direct parent of the new node. Then v = v.p moves to 12823.

Step 7 - Recompute node 12823

```
UPDATE(12823):
  subtreeProduced = 50 + 90 = 140
  subtreeEaten    = 30 + 35 =  65
  subtreeTossed   = 10 +  5 =  15
  subtreeCount    =  1 +  2 =   3

                [12823] (BLACK)  Psum=140
                       \
                     12825 (RED) Psum=90
                             \
                           12826 (RED) Psum=50
```

The right subtree of 12823 now includes both 12825 and 12826. After 12823, v moves to T.nil and the loop stops. Because a red-black tree has height O(log n), updating this ancestor path costs O(log n).

Step 8 - Red-Black Tree insertion fixup

```
BEFORE:                              AFTER:

   12823 (BLACK)                        12825 (BLACK)
```

```
          \                          /          \
        12825 (RED)             12823 (RED)      12826 (RED)
               \
            12826 (RED)
```

```
Fix: left-rotate at 12823, and recolor as shown.
```

The 12825 -> 12826 edge is RED-RED, so the tree violates the RB-tree rules. In this right-right case, a left rotation at 12823 plus recoloring restores the properties.

```
After rotation, update the node that moved DOWN first:
UPDATE(12823): P=50, E=30, T=10, Count=1

Then update the node that moved UP:
UPDATE(12825): P=140, E=65, T=15, Count=3
```

A rotation changes subtree membership, which is why both nodes must have their augmented information recomputed.

Step 9 - Existing date: add EATEN = 30

```
UPDATE_FOOD_INVENTORY(T, EATEN, 30, 12826)

z = RB-SEARCH(T, 12826)     // date exists
z.eaten = z.eaten + 30     // 0 + 30 = 30

v = z
while v != T.nil:
    UPDATE(v)
    v = v.p
```

```
               12825 (BLACK) Esub=95
               /              \
       12823 (RED)          12826 (RED)
       own E=30             own E=30
```

We update node 12826 and then its parent 12825. We do not create another node and do not need insertion fixup.

Step 10 - Existing date: add TOSSED = 10

```
UPDATE_FOOD_INVENTORY(T, TOSSED, 10, 12826)
z.tossed = 0 + 10 = 10
UPDATE(12826); UPDATE(12825)

Final tree:
                    12825 (BLACK)
                   /             \
           12823 (RED)        12826 (RED)

Own amounts:
  12823: P=50 E=30 T=10
  12825: P=40 E=35 T=5
  12826: P=50 E=30 T=10

Subtree values:
  12823: P=50  E=30 T=10 Count=1
  12826: P=50  E=30 T=10 Count=1
  12825: P=140 E=95 T=25 Count=3
```

The final tree is balanced and all the stored subtree totals are correct.

5. REPORT(T, x, d1, d2)

To find the average of a food type x over a date range [d1, d2], compute totals up to and including d2 and subtract totals strictly before d1. The difference gives the sum and number of reported dates inside the range.

VALUE(z,x) is the daily amount of x stored at z. SUBTREE-SUM(z,x) is the previously computed subtree total for food type x.

PREFIX helper pseudocode

```
PREFIX(T, x, d, inclusive):
    z = T.root
    total = 0
    count = 0

    while z != T.nil:
        if z.date < d or
           (inclusive and z.date == d):

            total = total
                + SUBTREE-SUM(z.left, x)
                + VALUE(z, x)

            count = count
                + z.left.subtreeCount + 1

            z = z.right
        else:
            z = z.left

    return (total, count)
```

REPORT pseudocode

```
REPORT(T, x, d1, d2):
    (sum2, count2) = PREFIX(T, x, d2, TRUE)
    (sum1, count1) = PREFIX(T, x, d1, FALSE)

    total = sum2 - sum1
    count = count2 - count1

    if count == 0:
        return 0

    return total / count
```

Here, returning 0 for an empty date range is our chosen convention. Count means reported dates (tree nodes), including a date on which food type x has value zero.

6. Visual walkthrough of REPORT

```
REPORT(T, PRODUCED, 12824, 12826)

                  12825 (BLACK)
                 /             \
          12823 (RED)      12826 (RED)
          produced=50      produced=50
```

```
Root (12825) produced=40
```

Step 1 - PREFIX(T, PRODUCED, 12826, TRUE)

Start at root 12825. Because 12825 <= 12826, add the whole left subtree (12823) and the root itself: total = 50 + 40 = 90 and count = 1 + 1 = 2. Then move right to 12826.

At 12826, equality is included. Add its 50: total = 140, count = 3. Return (sum2, count2) = (140, 3).

```
            12825
           /     \
     [12823]      12826
      add 50        add 50
            + root's own 40
       sum2 = 140, count2 = 3
```

Step 2 - PREFIX(T, PRODUCED, 12824, FALSE)

Start at 12825. It is greater than 12824, so go left. At 12823, the date is strictly smaller than 12824; add produced=50 and count 1. Return (sum1, count1) = (50, 1).

```
            12825
            /     \
       [12823]    12826
        add 50

      sum1 = 50, count1 = 1
```

Step 3 - Subtract the prefixes

```
total = sum2 - sum1 = 140 - 50 = 90
count = count2 - count1 =   3 -  1 =  2
```

Exactly two reported dates are in [12824, 12826]: dates 12825 and 12826.

Step 4 - Divide by the count

```
Average = total / count = 90 / 2 = 45

REPORT(PRODUCED, 12824, 12826) RETURNS 45.
```

Date 12824 does not count because no report exists for that date. A reported date with produced=0 would still count.

7. Worst-case time complexity - exam-ready

UPDATE_FOOD_INVENTORY: O(log n)

RB-SEARCH takes O(log n) because the tree height is O(log n). For an existing date, adding q is O(1) and UPDATE is applied only to the O(log n) ancestors. For a new date, RB-BST-INSERT is O(log n), ancestor updates are O(log n), and RB-INSERT-FIXUP is O(log n). Each local UPDATE is O(1), including updates after a rotation. Overall: O(log n) worst case.

REPORT: O(log n)

Each PREFIX follows just one downward path in the RB tree, using saved subtree sums and counts without scanning every node. One PREFIX costs O(log n). REPORT runs two PREFIX queries and O(1)

arithmetic, so O(log n) + O(log n) + O(1) = O(log n) worst case.

8. Quick revision table

OPERATION	MAIN IDEA	WORST CASE
RB-SEARCH	Find the date	O(log n)
NEW-NODE	Create a node	O(1)
Initialize fields	Set own and subtree totals	O(1)
RB-BST-INSERT	Insert key by date	O(log n)
UPDATE(v)	Recompute one node	O(1)
Update ancestor path	Recompute all ancestors	O(log n)
RB-INSERT-FIXUP	Restore RB properties	O(log n)
PREFIX	Get prefix sum and count	O(log n)
REPORT	Two prefixes and a division	O(log n)

EXAM TAKEAWAY: Store subtree sums for produced, eaten and tossed, plus a count of dates, in an augmented Red-Black Tree. That supports both updating the history and calculating the average over a date range in O(log n) worst-case time.